

CS675

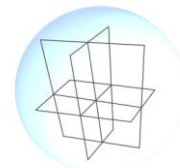
Computer Graphics

Visibility

Parag Chaudhuri



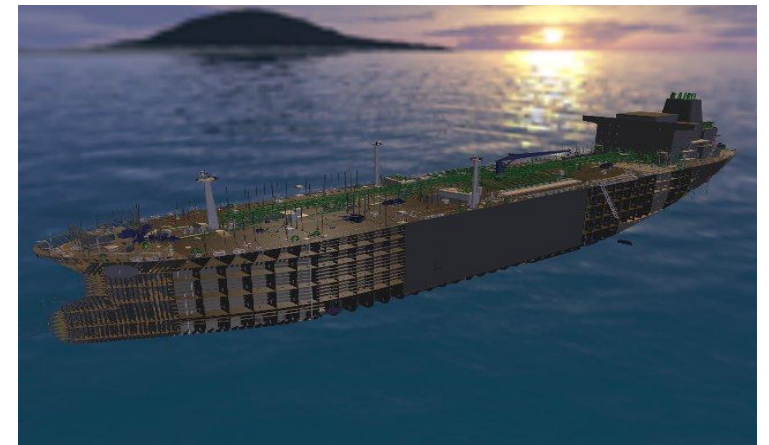
Department of Computer Science and Engineering
IIT Bombay



ViGIL
VISION, GRAPHICS,
IMAGING & LEARNING

Visibility

- What is visible?
 - Which objects are visible?
 - Which pixels(fragments) to render?
- Why check for visibility?
 - Efficiency
 - Correctness?
 - Disambiguation



The Double Eagle Tanker:

4GB of data, 82 M Triangles

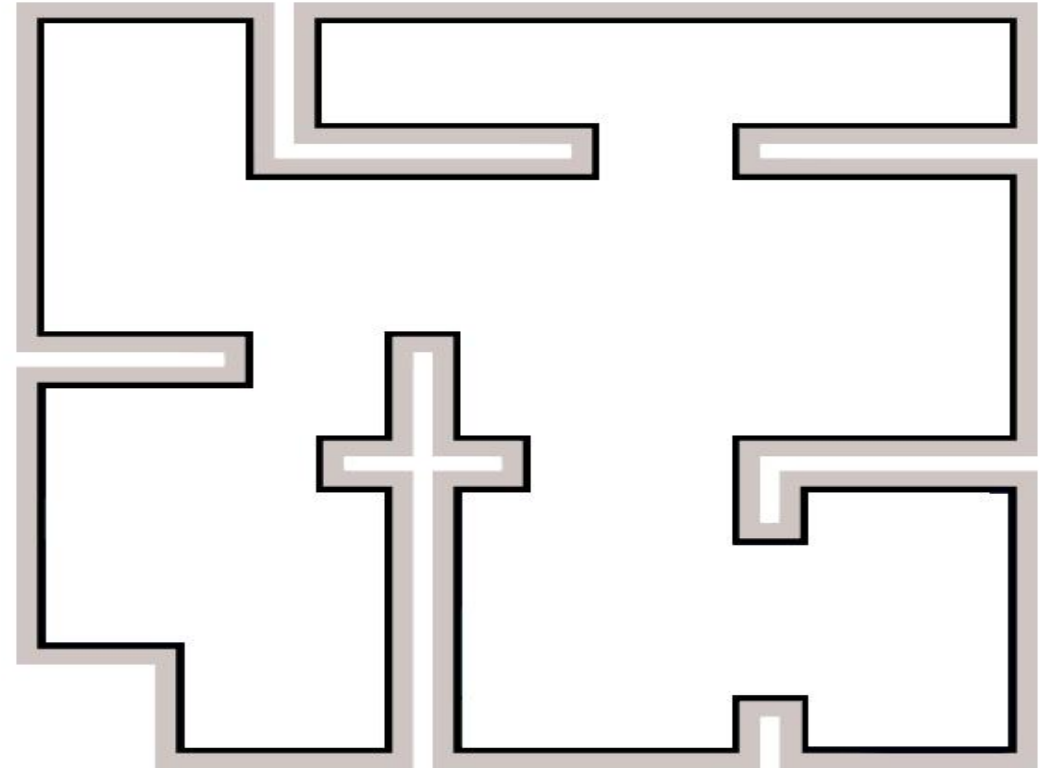
From:

<http://www.cs.unc.edu/~geom/hardware/#Vis>

Simple question

- The *art gallery* problem:

Given a planar art gallery, what is the minimum number of guards that need to be placed at the corners (but inside the gallery) so that every part of the gallery is visible to at least one guard.



Not so simple answer

- NP – Complete!
- Upper bound: $\text{floor}(N/3)$ for a simple polygon with N vertices.
- Message: Determining visibility is not always easy.

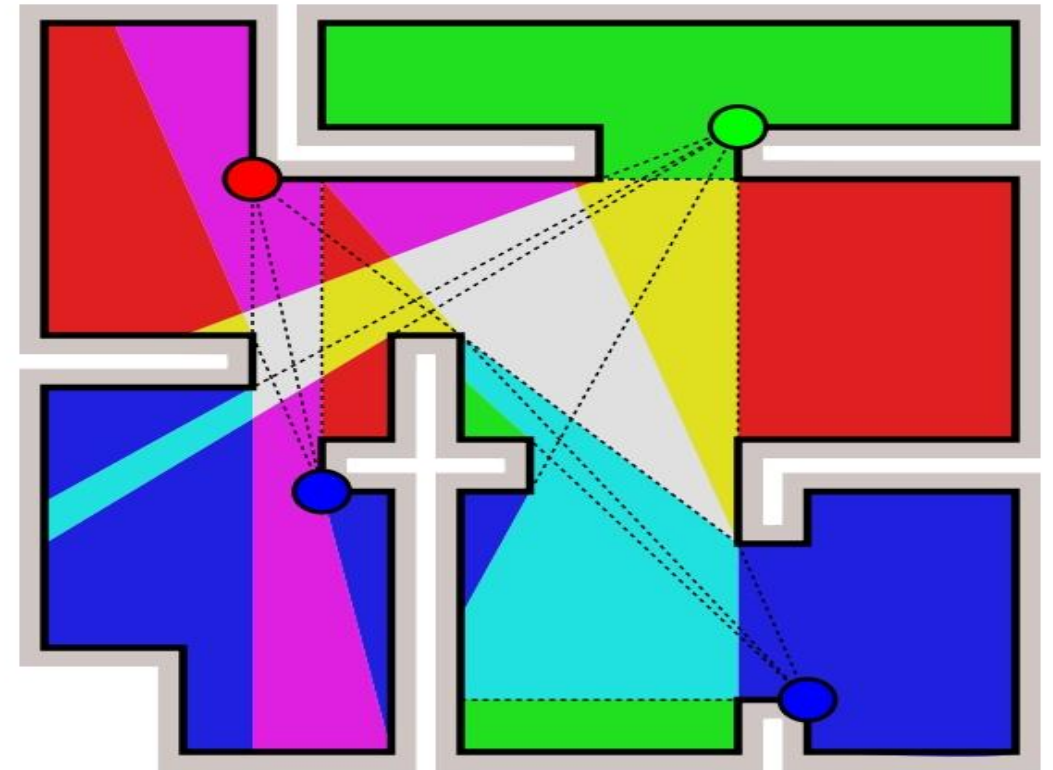


Image Space Methods

Visibility

The Image Space problem formulation

```
for (each pixel in the rendered image)
{
    determine the object closest to the viewer that is
        intercepted by the projector (ray) through the pixel;
    draw the pixel in the appropriate color;
}
```

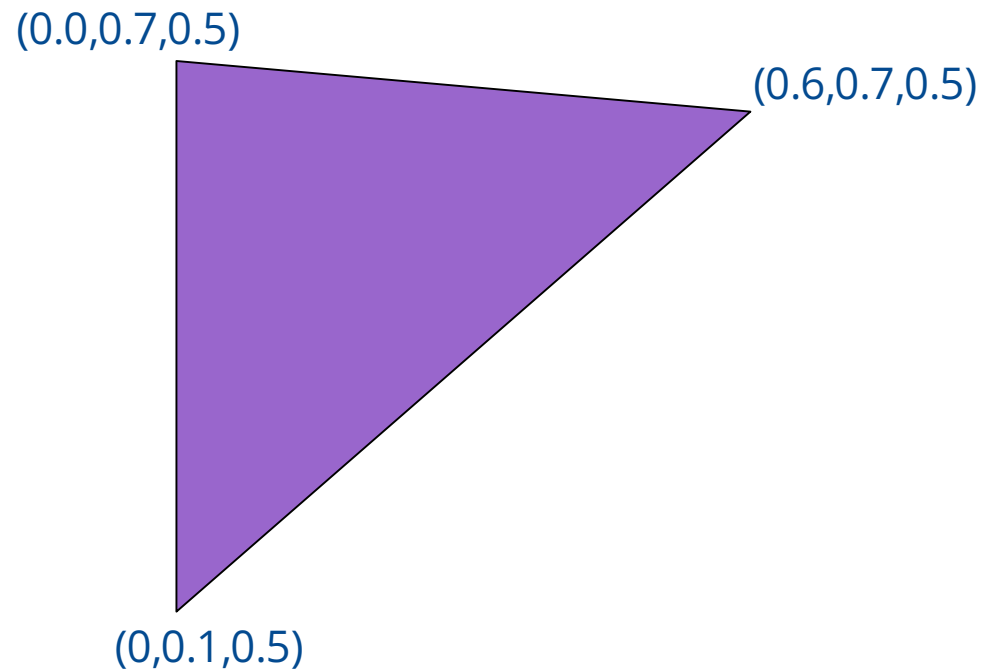
- Worst case complexity: np
- n : number of objects, p : number of pixels

Z-Buffer Algorithm

- Initialize the z-buffer to the max Z value.
- `glClear, glDepthRange`

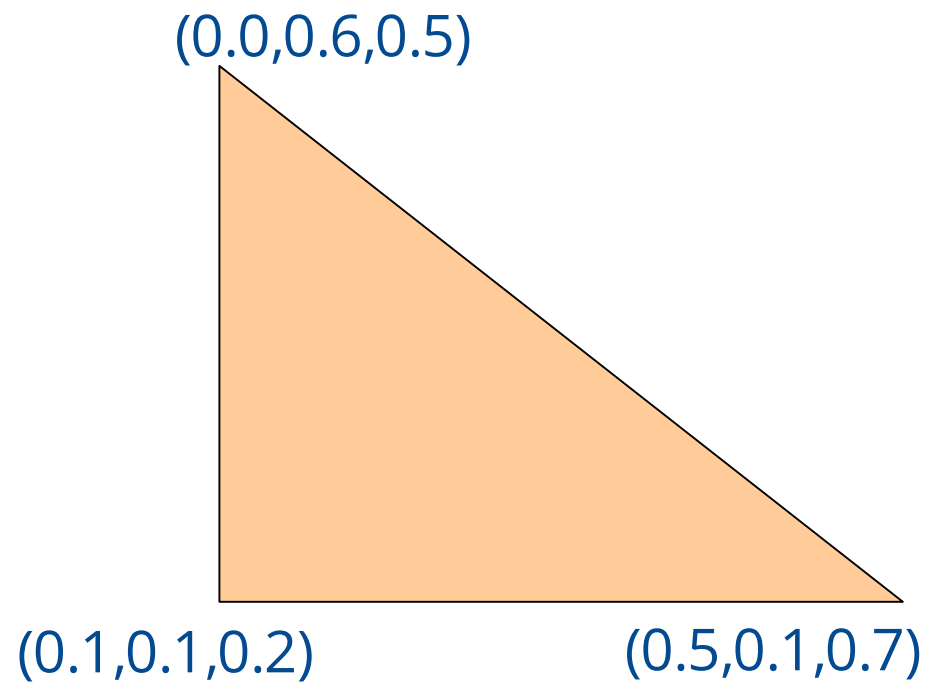
[illegible]

Z-Buffer Algorithm



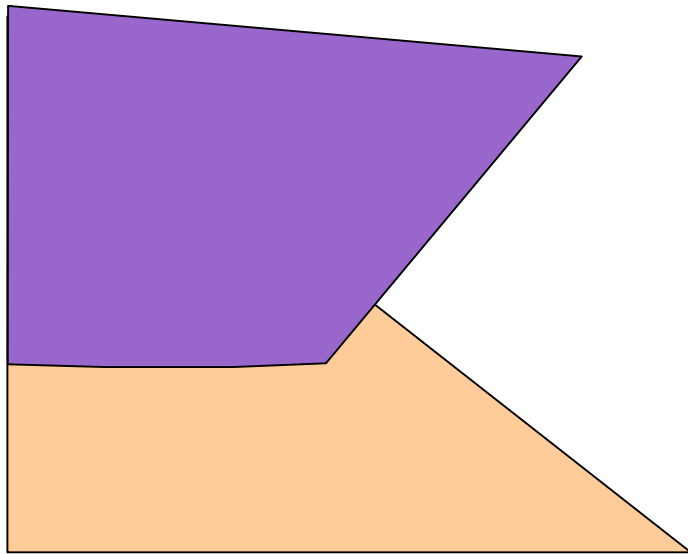
1	1	1	1	1	1	1	1
1	0.5	0.5	1	1	1	1	1
1	0.5	0.5	0.5	0.5	0.5	1	1
1	0.5	0.5	0.5	0.5	1	1	1
1	0.5	0.5	0.5	1	1	1	1
1	0.5	0.5	1	1	1	1	1
1	0.5	0.5	1	1	1	1	1
1	0.5	1	1	1	1	1	1

Z-Buffer Algorithm



1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	0.5	0.5	1	1	1	1	1
1	0.5	0.5	1	1	1	1	1
1	0.5	0.5	0.5	1	1	1	1
1	0.4	0.5	0.6	0.7	1	1	1
1	0.3	0.4	0.5	0.6	0.7	1	1
1	0.2	0.3	0.4	0.5	0.6	0.7	1

Z-Buffer Algorithm



1	1	1	1	1	1	1	1
1	0.5	0.5	1	1	1	1	1
1	0.5	0.5	0.5	0.5	0.5	1	1
1	0.5	0.5	0.5	0.5	1	1	1
1	0.5	0.5	0.5	1	1	1	1
1	0.4	0.5	0.6	0.7	1	1	1
1	0.3	0.4	0.5	0.6	0.7	1	1
1	0.2	0.3	0.4	0.5	0.6	0.7	1

Note that almost everywhere the result is independent of the order of drawing these polygons. Except at the pixels where the depth may be the same (this is very unlikely).

Z-Buffer Algorithm

Initialize

- `zbuf[i, j]=MAX_DEPTH`
- `cbuf[i, j]=BACKGROUND_COLOR`

for (each scan converted polygon)

{

- Find pseudodepth, z , of polygon at pixel (x, y) with color c
- If $(z < \text{zbuf}[i, j])$ { `zbuf[i, j] = z; cbuf[i, j] = c;`}

}

Z-Buffer Algorithm

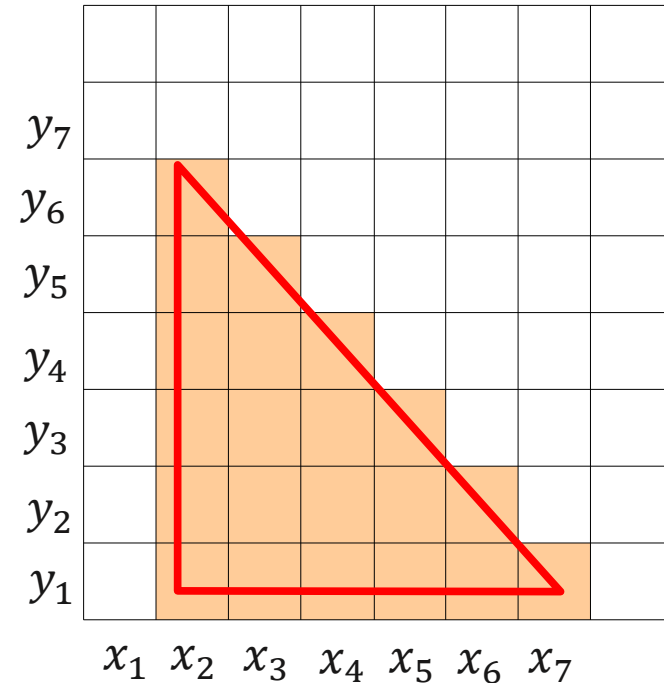
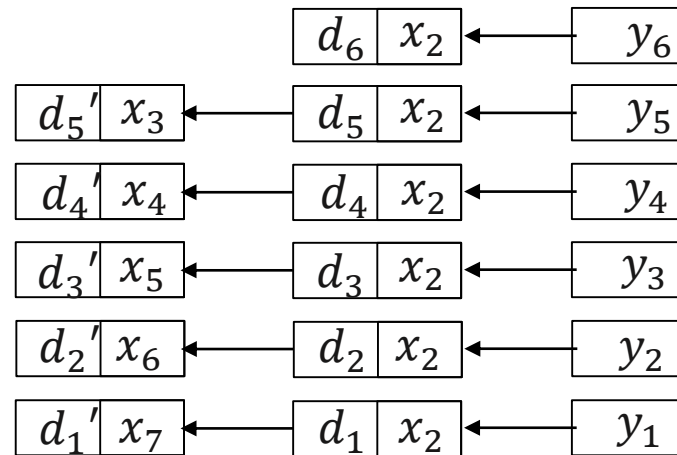
- Advantages
 - Simple, Accurate (modulo non-linear z-mapping).
 - Independent of order of drawing polygons.
- Disadvantages
 - Memory (not an issue these days).
 - Wasted computation when over-writing distant points
- Complexity
 - Time: $O(nm \cdot k)$ – $n \times m$ pixels, k polygons
 - Space: $O(nm \cdot b)$ – $n \times m$ pixels, b bytes precision per pixel

Z-Buffer Algorithm and Scan Conversion

- Construct the active edge list AEL for every scanline.
- Interpolate the pseudodepth, d , for each active edge.

```

for each edge  $[(x_i, y_i, d_i), (x_j, y_j, d_j)]$  with  $y_i < y_j$ 
{
     $x = x_i, d = d_i, \Delta x = \frac{(x_j - x_i)}{(y_j - y_i)}$  and  $\Delta d = \frac{(d_j - d_i)}{(y_j - y_i)}$ 
    for ( $y = y_i; y < y_j; y++$ )
    {
        insert  $(x, d)$  into the AEL of scanline  $y$ 
        such that it is sorted on the  $x$  values
         $x = x + \Delta x, d = d + \Delta d$ 
    }
}
    
```

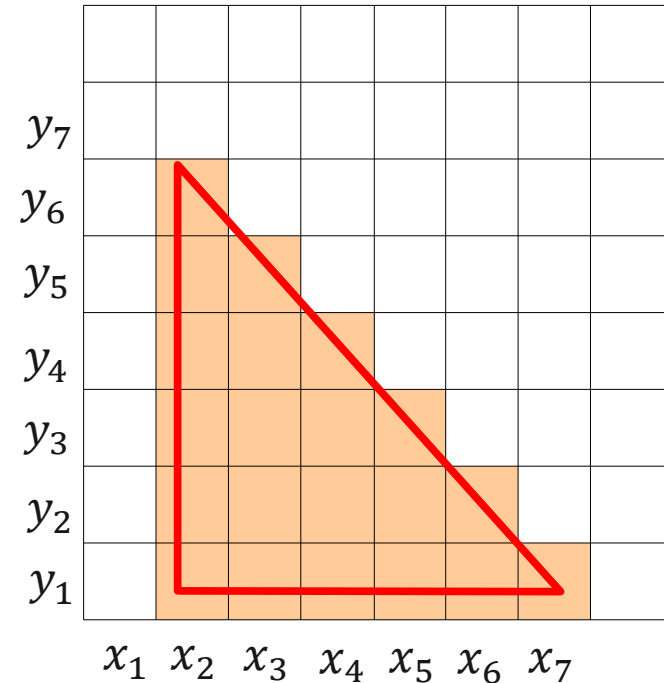
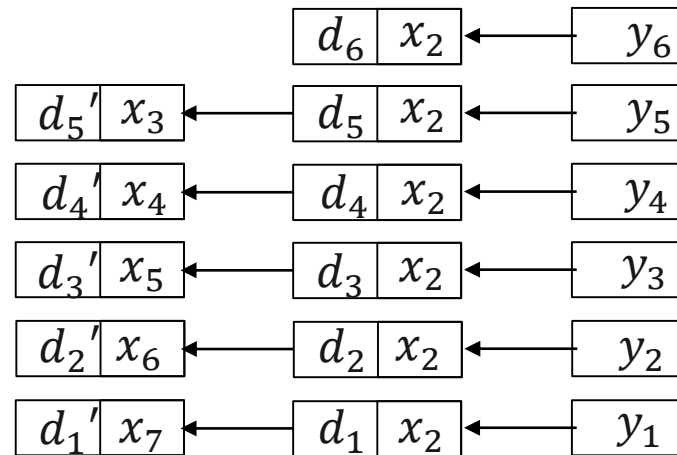


Z-Buffer Algorithm and Scan Conversion

- Construct the active edge list AEL for every scanline.
- Interpolate the pseudodepth, d , for each active edge.
- Interpolate the d along the scan line.

```

cbuf[i,j] = BACKGROUND_COLOR
zbuf[i,j] = MAX_DEPTH,  $\forall 0 \leq i < N, 0 \leq j < M$ 
 $y_{min} = \min(y_A, y_B, y_C), y_{max} = \max(y_A, y_B, y_C)$ 
for ( $y = y_{min}; y \leq y_{max}; y++$ )
{
  get ( $x_p, d_p$ ) and ( $x_q, d_q$ ) from the AEL with  $x_p < x_q$ 
   $\Delta d = (d_p - d_q) / (x_p - x_q)$ 
}
    
```



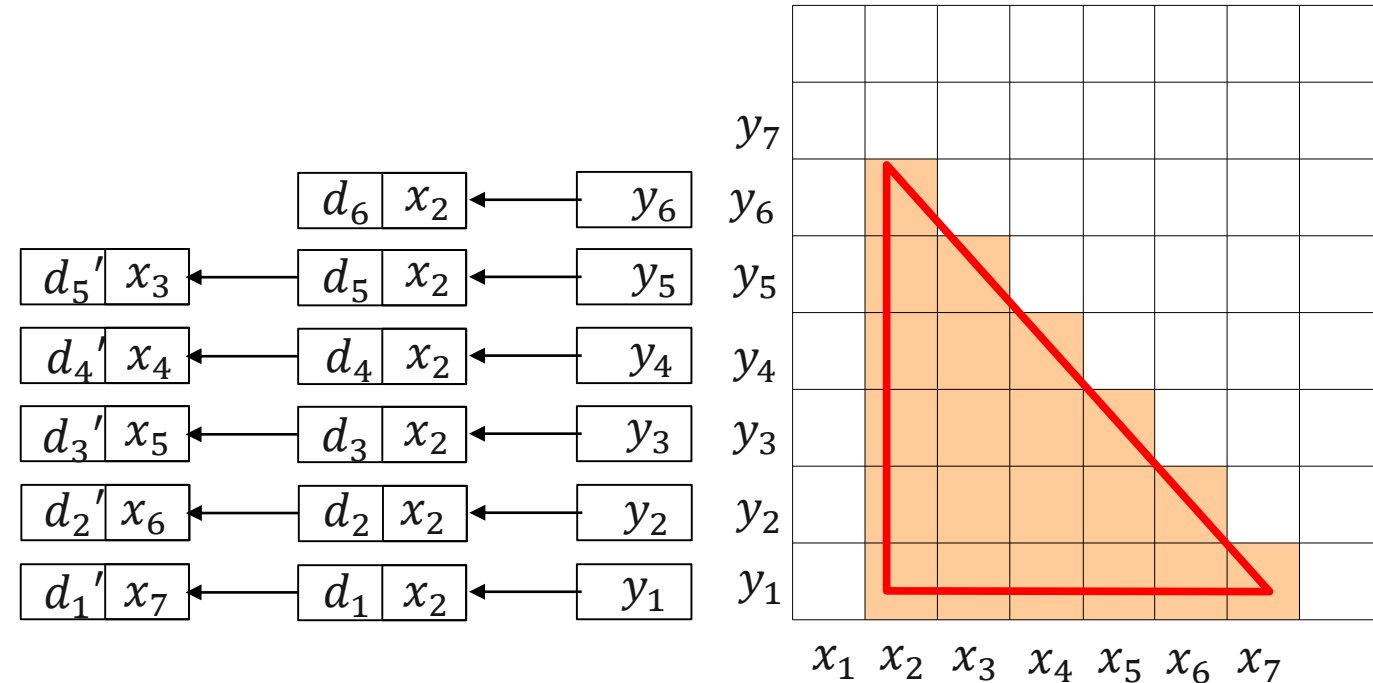
Z-Buffer Algorithm and Scan Conversion

- Construct the active edge list AEL for every scanline.
- Interpolate the pseudodepth, d , for each active edge.
- Interpolate the d along the scan line.

```

cbuf[i,j] = BACKGROUND_COLOR
zbuf[i,j] = MAX_DEPTH,  $\forall 0 \leq i < N, 0 \leq j < M$ 
 $y_{min} = \min(y_A, y_B, y_C), y_{max} = \max(y_A, y_B, y_C)$ 
for ( $y = y_{min}; y \leq y_{max}; y++$ )
{
  get ( $x_p, d_p$ ) and ( $x_q, d_q$ ) from the AEL with  $x_p < x_q$ 
   $\Delta d = (d_p - d_q) / (x_p - x_q)$ 
  for ( $x = x_p, d = d_p; x \leq x_q; x++, d = d + \Delta d$ )
  {
    if ( $d < zbuf[x, y]$ ) {  $zbuf[x, y] = d, cbuf[x, y] = c$  }
  }
}

```



- Do the z-buffer depth test for each pixel and colour the ones that pass the test.
- The color c at a pixel is also interpolated along the scanline like the pseudodepth is

Object Space Methods

Visibility

The **Object Space** problem formulation

for (every *object* in the world)

{

 determine those parts of the object whose view is

 unobstructed by other parts of itself or any other object;

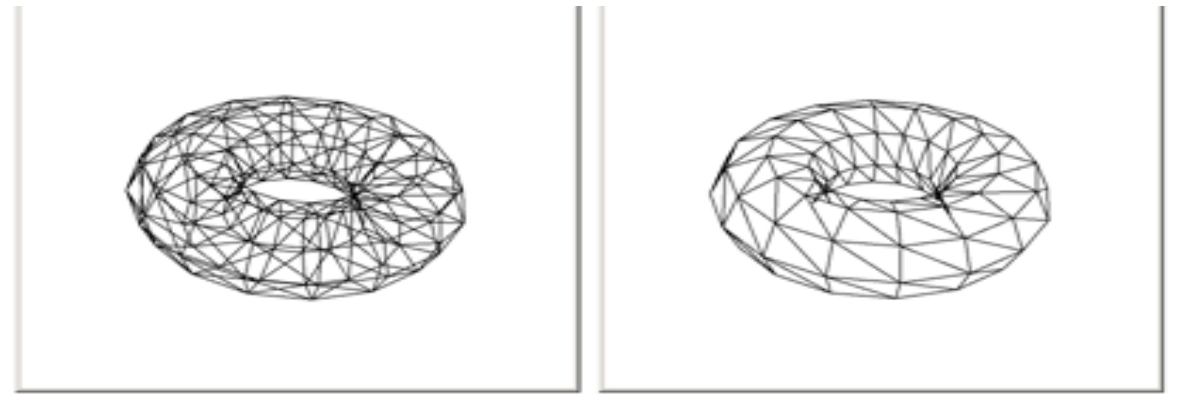
 draw those parts in the appropriate color;

}

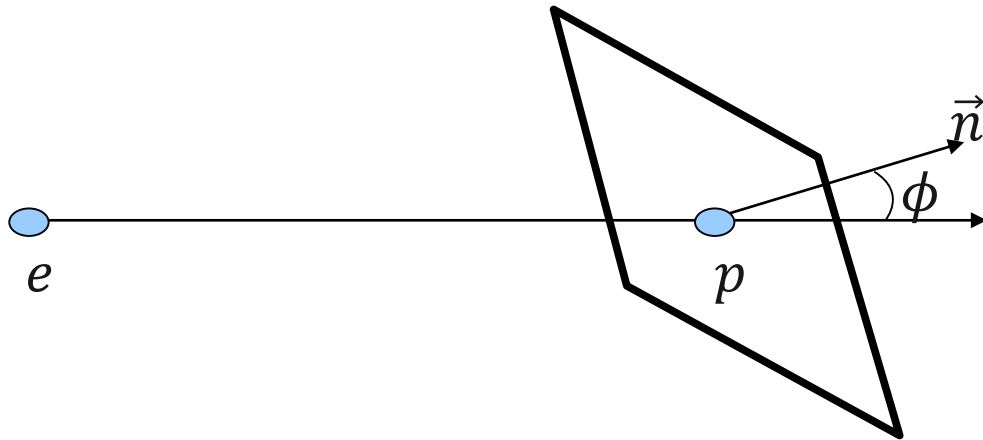
- Worst case complexity: n^2
- n : number of objects

Visibility

- Types of visibility computation we have seen:
 - Clipping – 2D and 3D
 - View-frustum clipping/culling
 - Backface culling

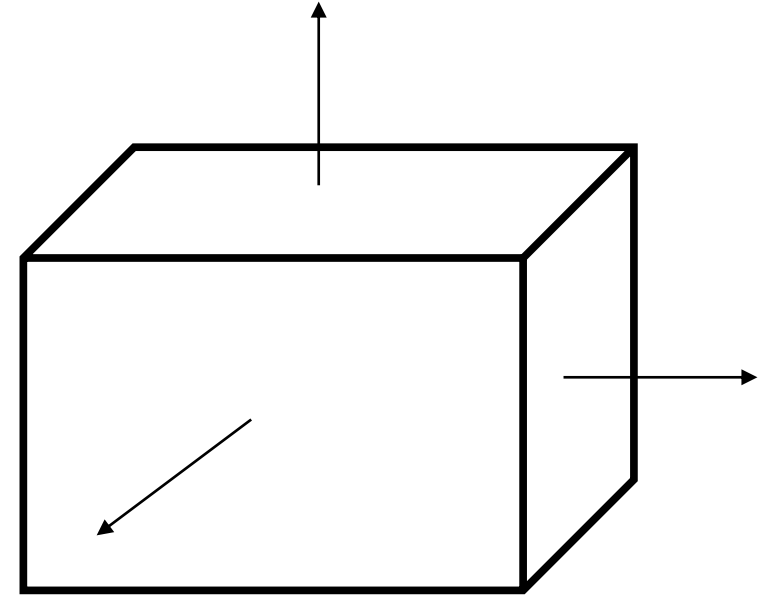


Backface Culling



$$(p - e) \cdot \vec{n} > 0 \quad \text{Cull}$$

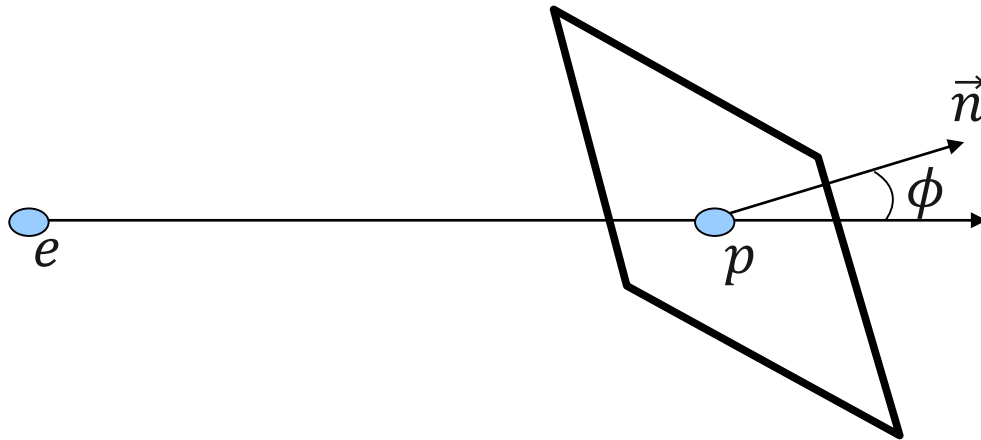
$$(p - e) \cdot \vec{n} \leq 0 \quad \text{Do Not Cull (may be visible)}$$



Simple Idea:

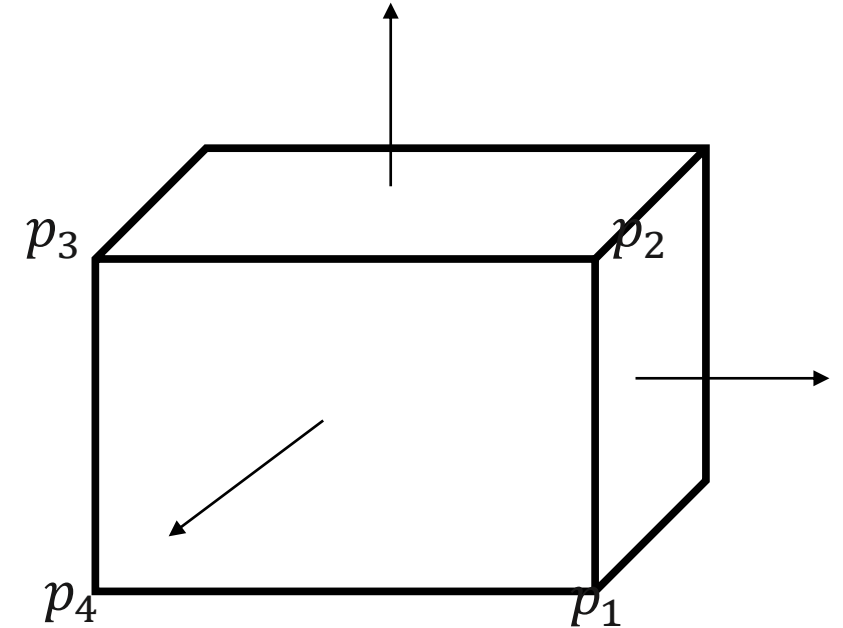
Discard surface patches that face away from the camera.

Backface Culling



If p_1, p_2, p_3, p_4 are the patch vertices in CCW order seen from outside then the outward facing normal is given by:

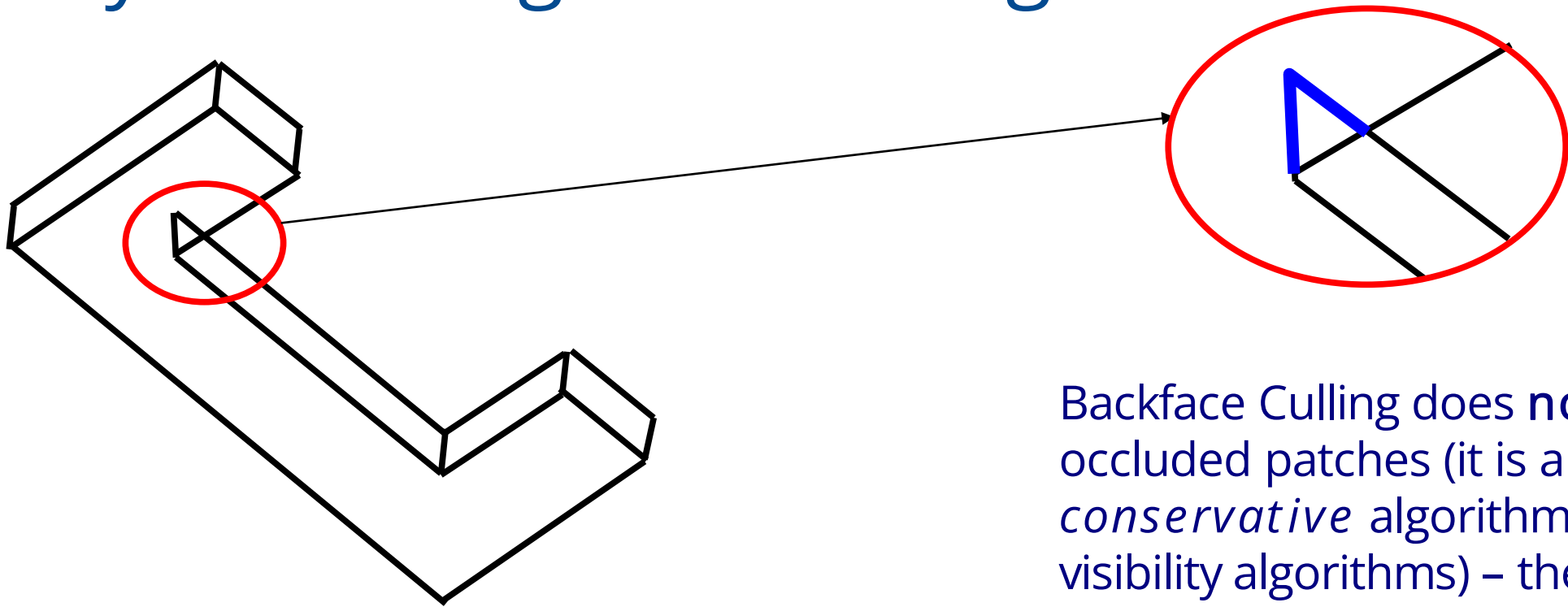
$$\vec{n} = (p_2 - p_1) \times (p_3 - p_1)$$



Compute the outward normals and do Backface culling in the WCS.

Backface Culling

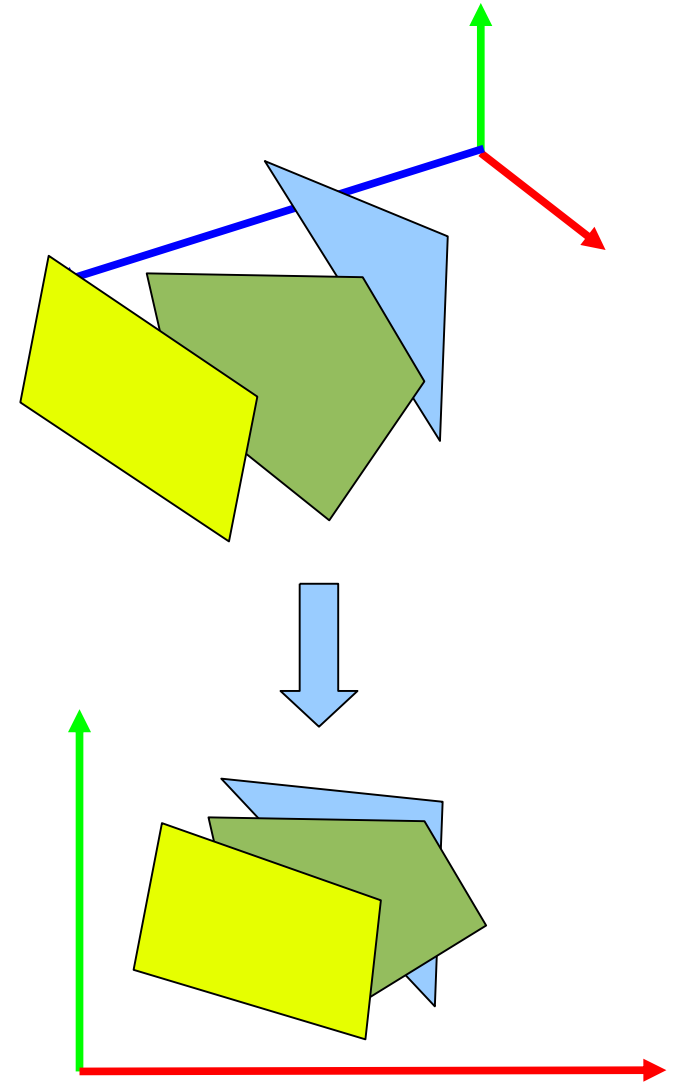
Only this culling is not enough



Backface Culling does **not** remove all occluded patches (it is a *conservative* algorithm, as are many visibility algorithms) – the example shown here is a case of *self* occlusion.

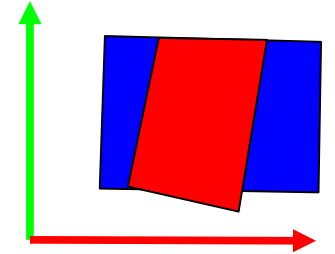
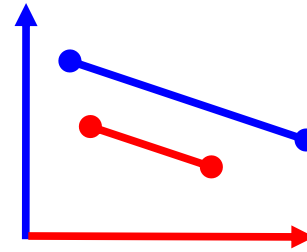
Painter's Algorithm

- Sort polygons in increasing order of depth.
- Draw the sorted list of polygons from back to front, i.e., from greatest depth to lesser depths.
- What happens when a polygon has vertices at different depths?



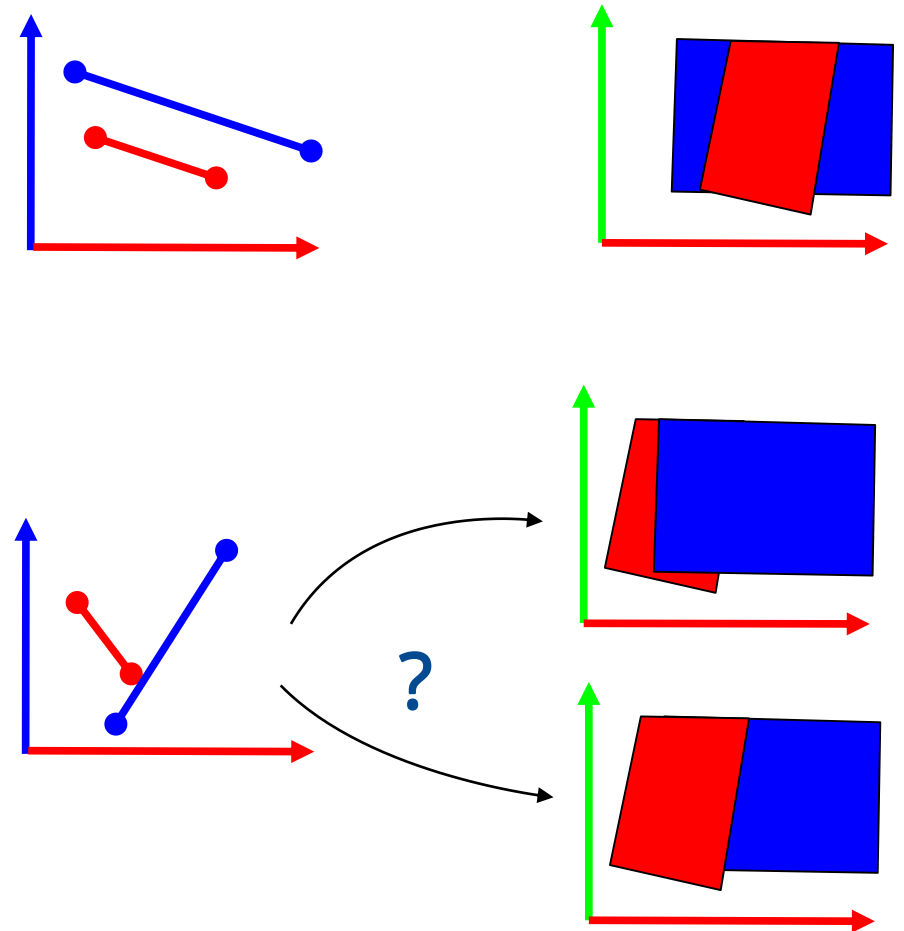
Painter's Algorithm

- Sort polygons in increasing order of depth.
- Draw the sorted list of polygons from back to front, i.e., from greatest depth to lesser depths.
- What happens when a polygon has vertices at different depths?
- Sort according to depth of farthest vertex.

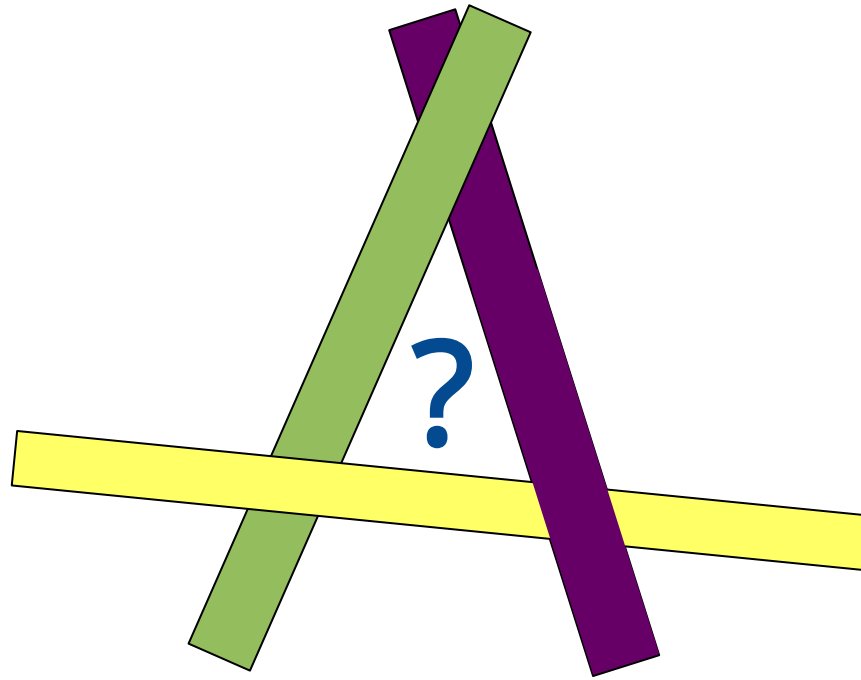


Painter's Algorithm

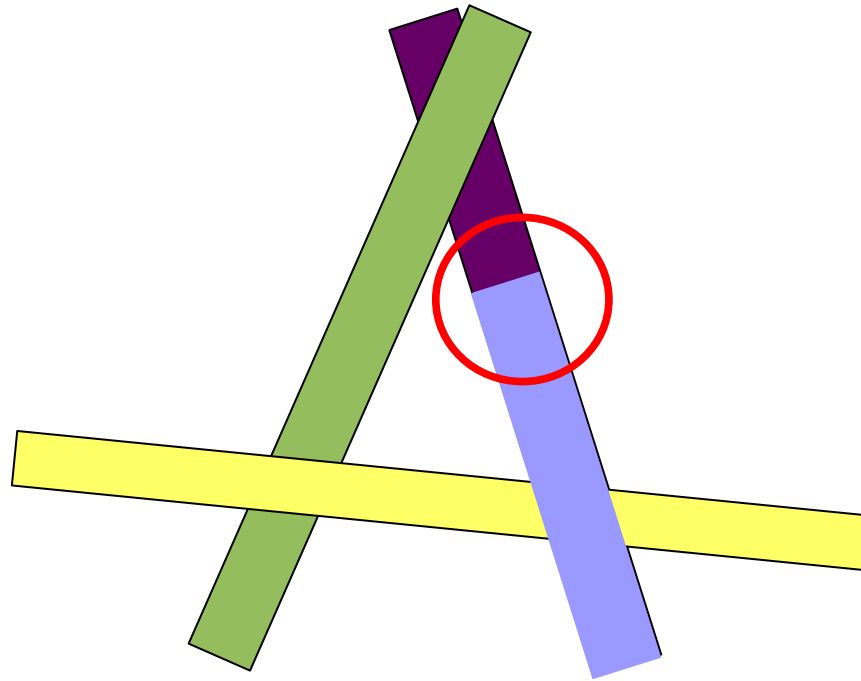
- Sort polygons in increasing order of depth.
- Draw the sorted list of polygons from back to front, i.e., from greatest depth to lesser depths.
- What happens when a polygon has vertices at different depths?
- Sort according to depth of farthest vertex.
- Does it always work?
- How often do we sort?



Painter's Algorithm

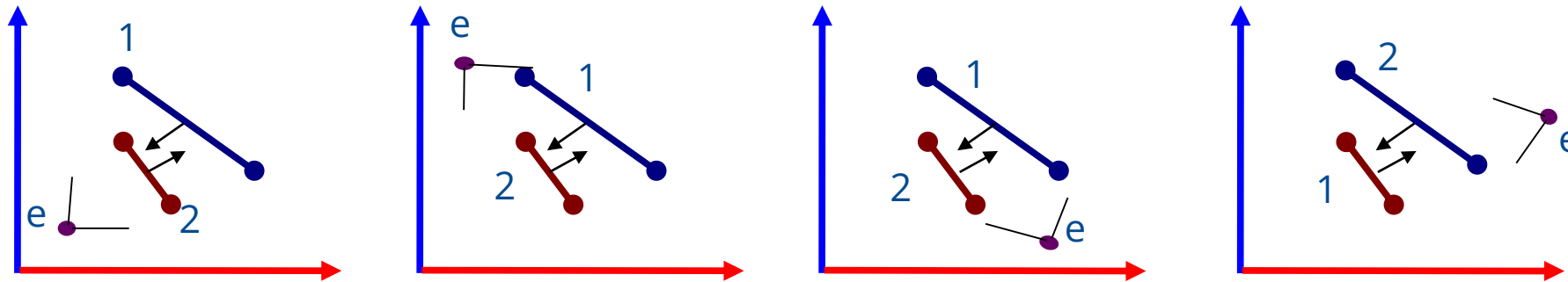


Painter's Algorithm



Space Subdivision/Partitioning

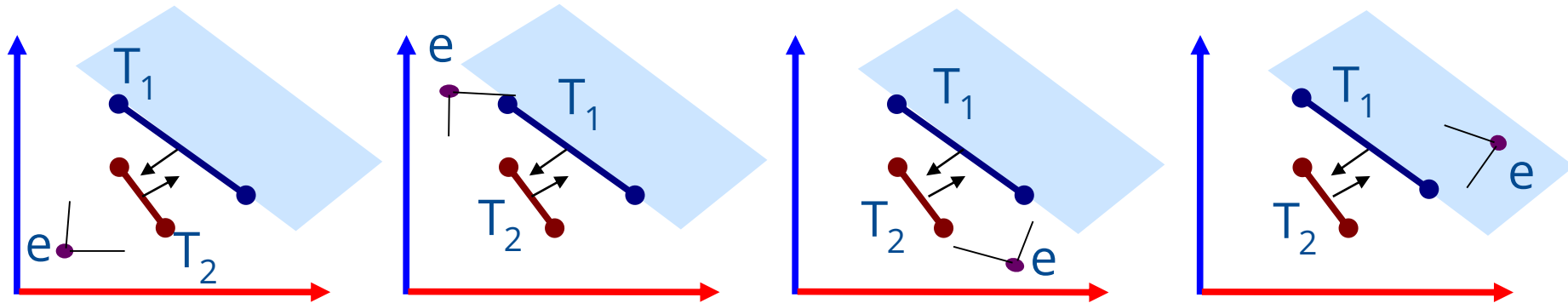
- Binary Space Partitioning (BSP) Trees



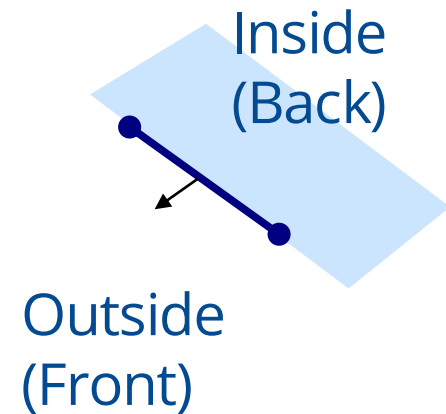
- Observe the correct order of drawing polygons as the eye moves

Space Subdivision/Partitioning

- Binary Space Partitioning (BSP) Trees

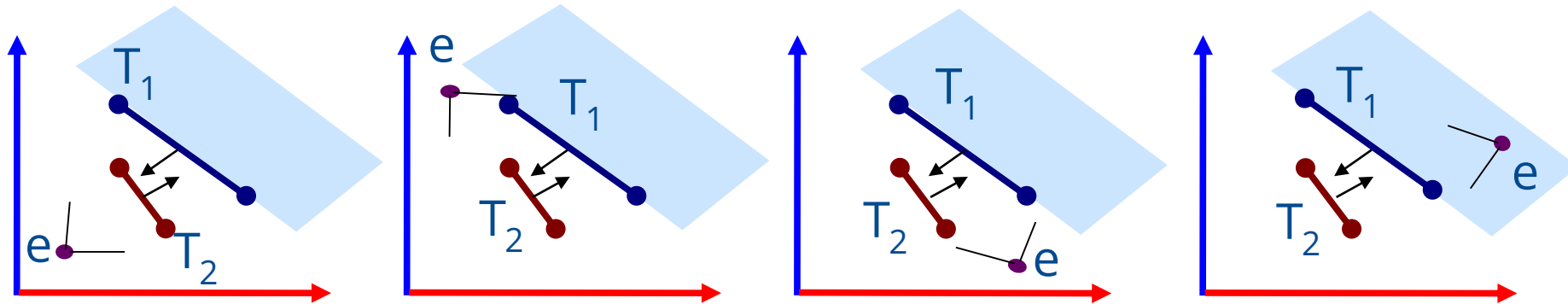


- If e and T_2 are on the same side of T_1
 - Draw T_1 and then draw T_2
- If e and T_2 are on different sides of T_1
 - Draw T_2 and then draw T_1

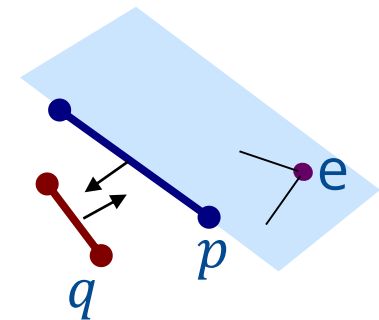


Space Subdivision/Partitioning

- Binary Space Partitioning (BSP) Trees

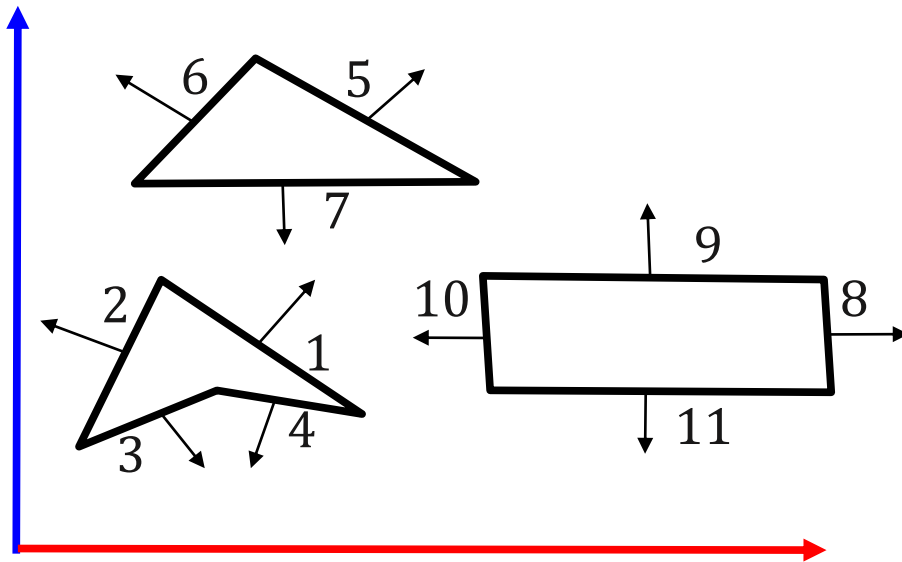


- If the implicit equation of the plane containing T_1
- is given by: $f(r) = (r - p) \cdot n$
- If $f(q) \cdot f(e) > 0$ then draw T_1 and then draw T_2
- If $f(q) \cdot f(e) < 0$ then draw T_2 and then draw T_1



BSP Trees

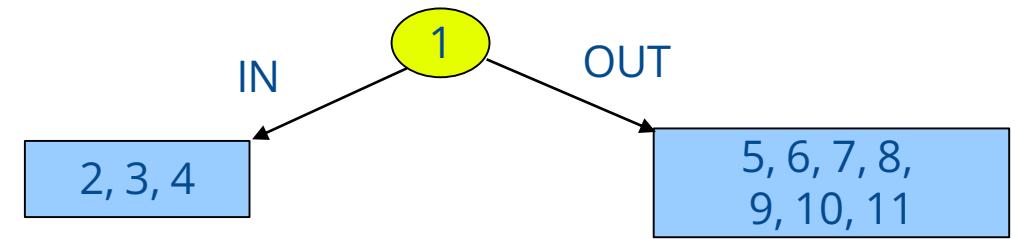
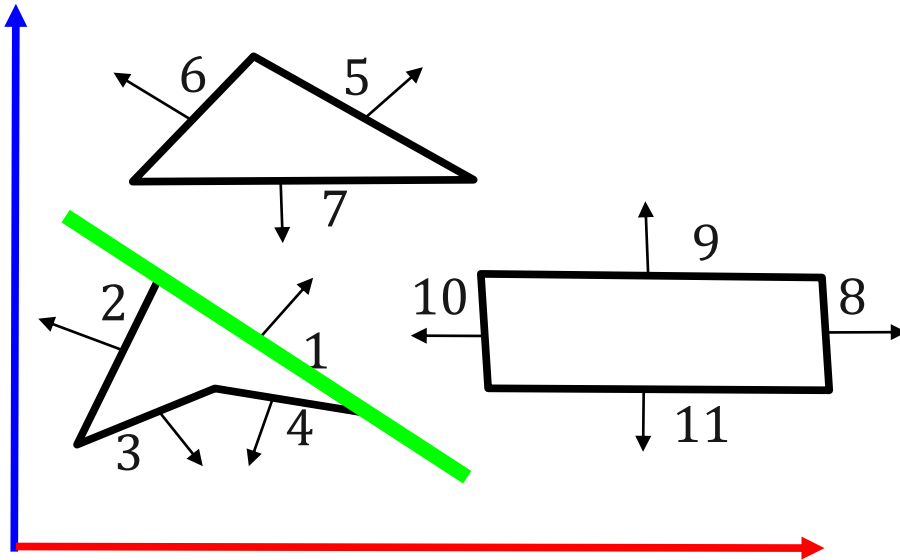
- BSP Tree is an efficient data structure for quickly determining the inside/outside relation between polygons and the camera position.



- Two Phases
 - Preprocessing: BSP Tree construction
(done once for a given scene)
 - Rendering: BSP Tree traversal
(done whenever the eye position changes)

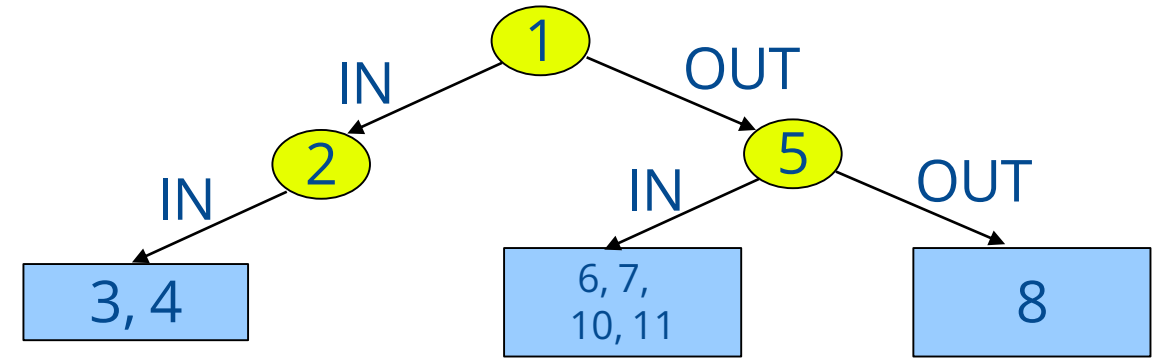
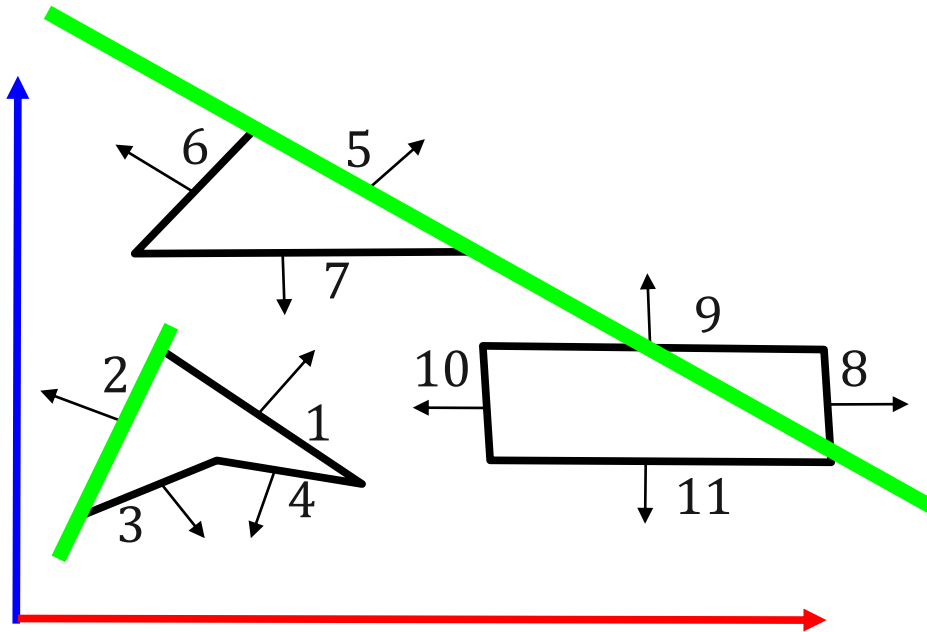
BSP Trees

Tree construction



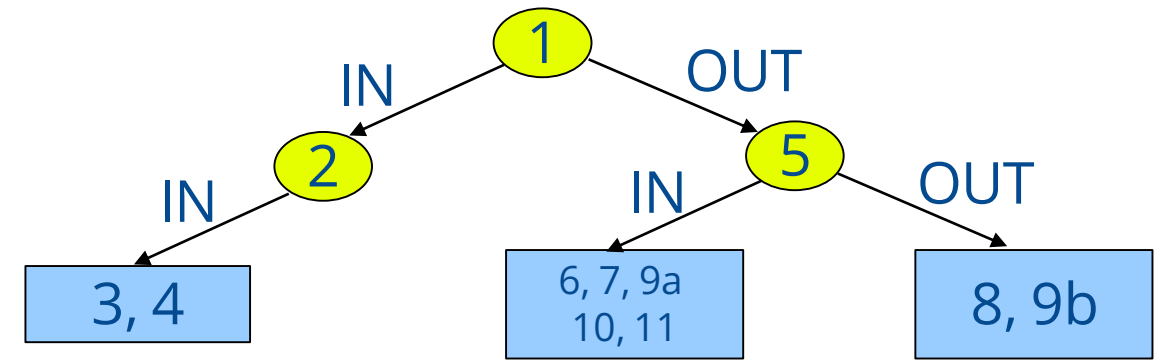
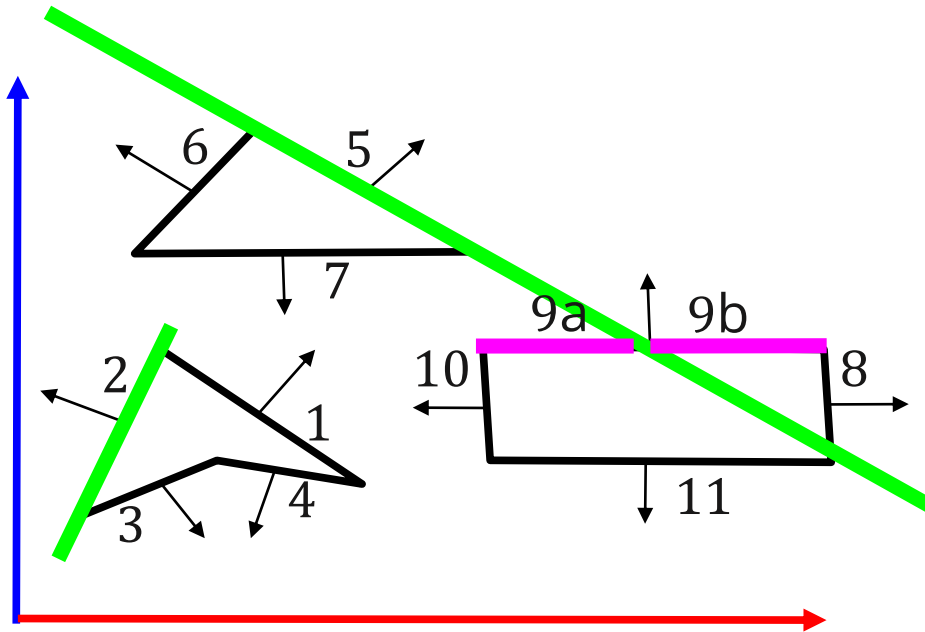
BSP Trees

Tree construction



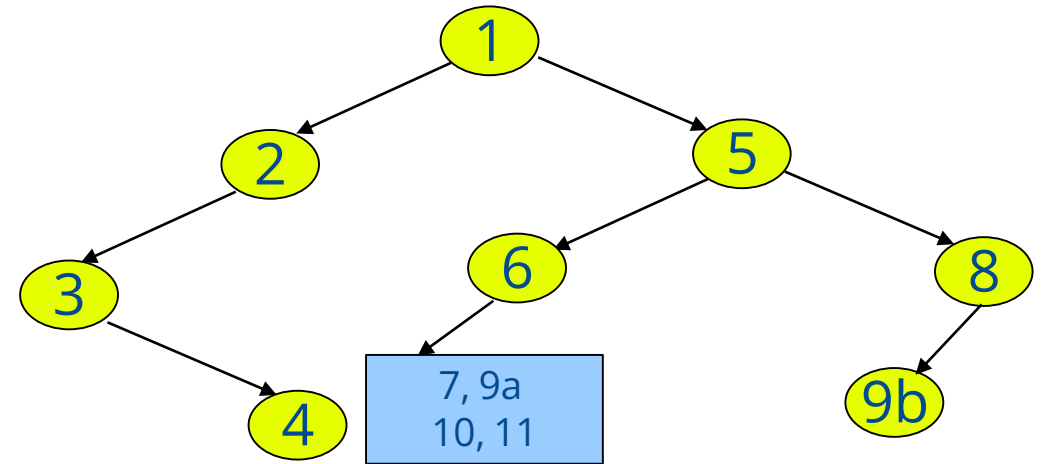
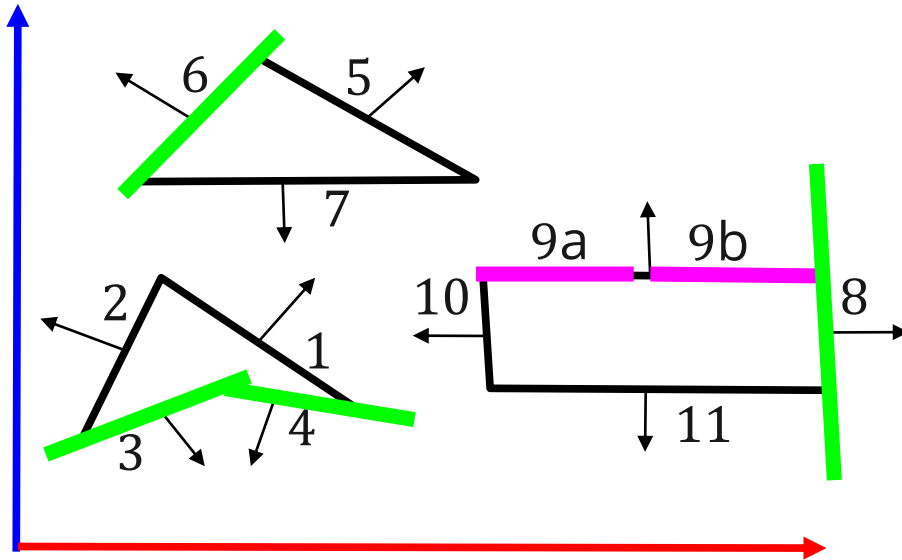
BSP Trees

Tree construction



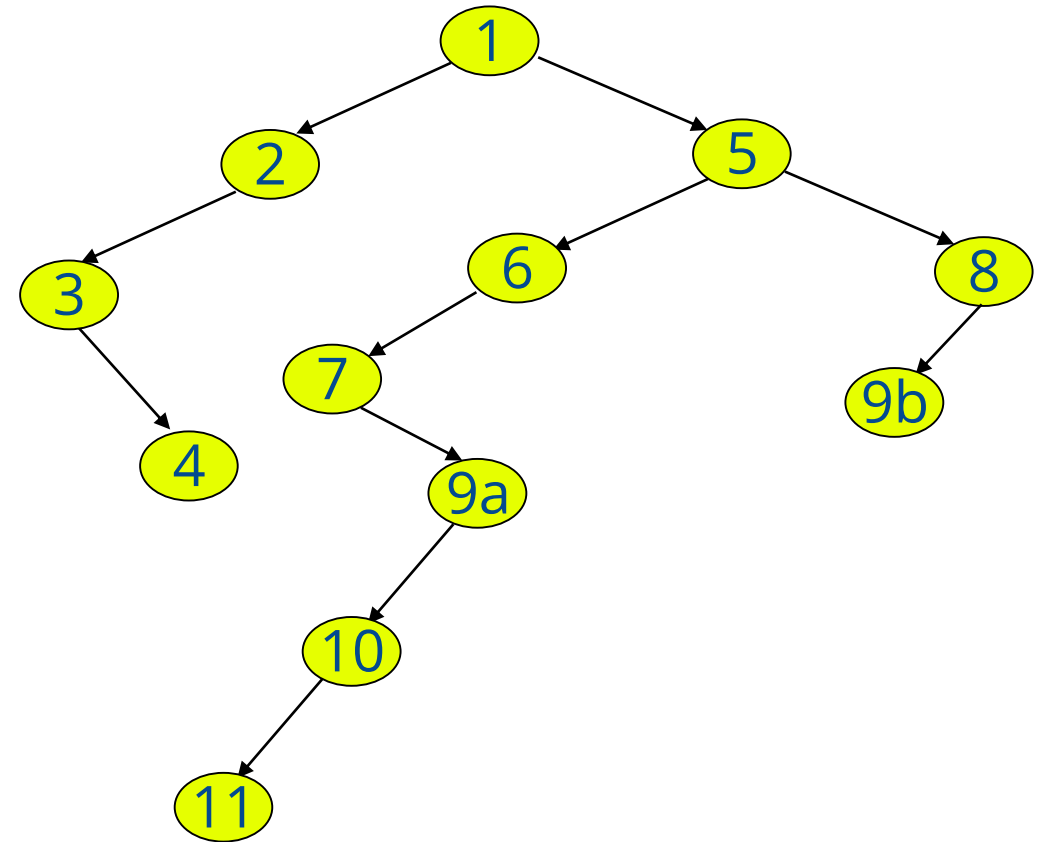
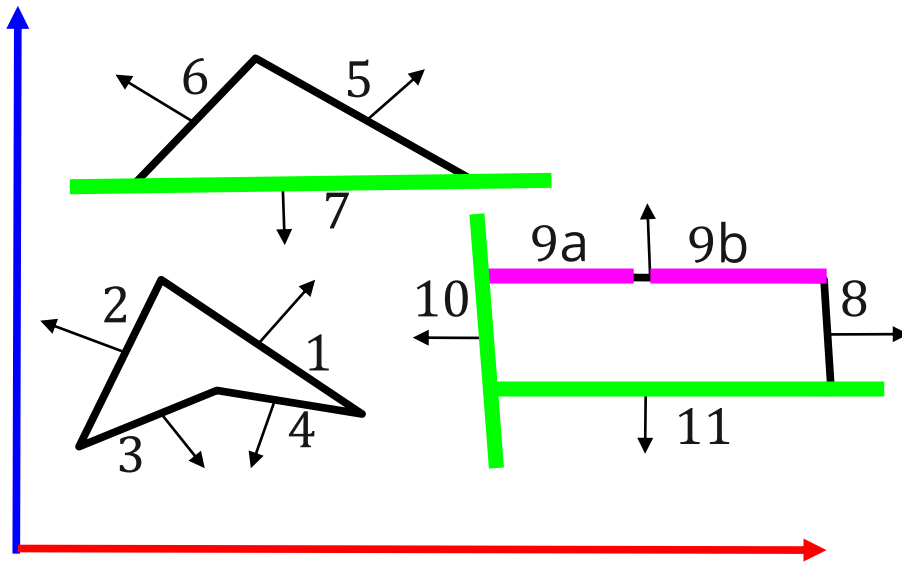
BSP Trees

Tree construction



BSP Trees

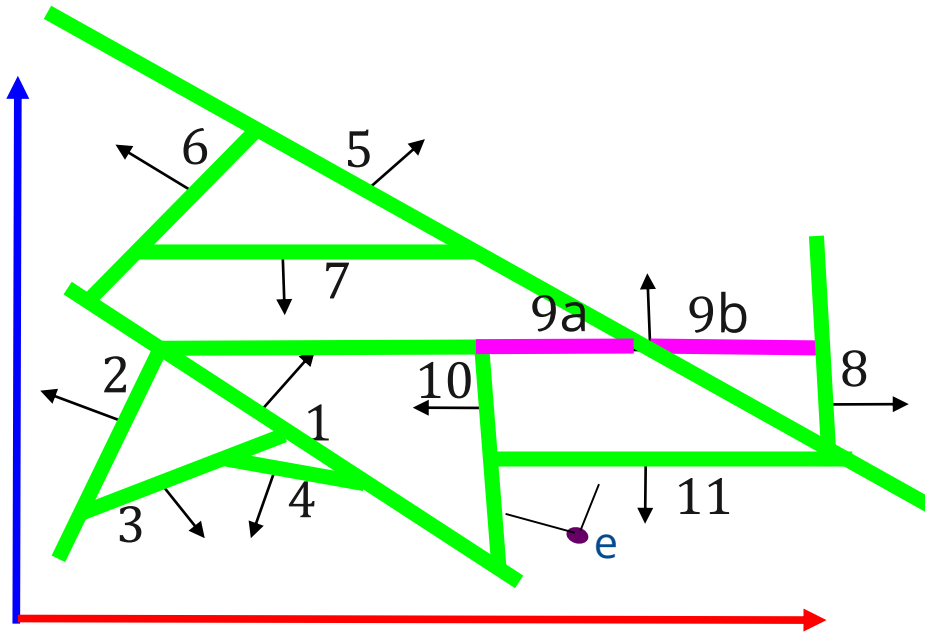
Tree construction



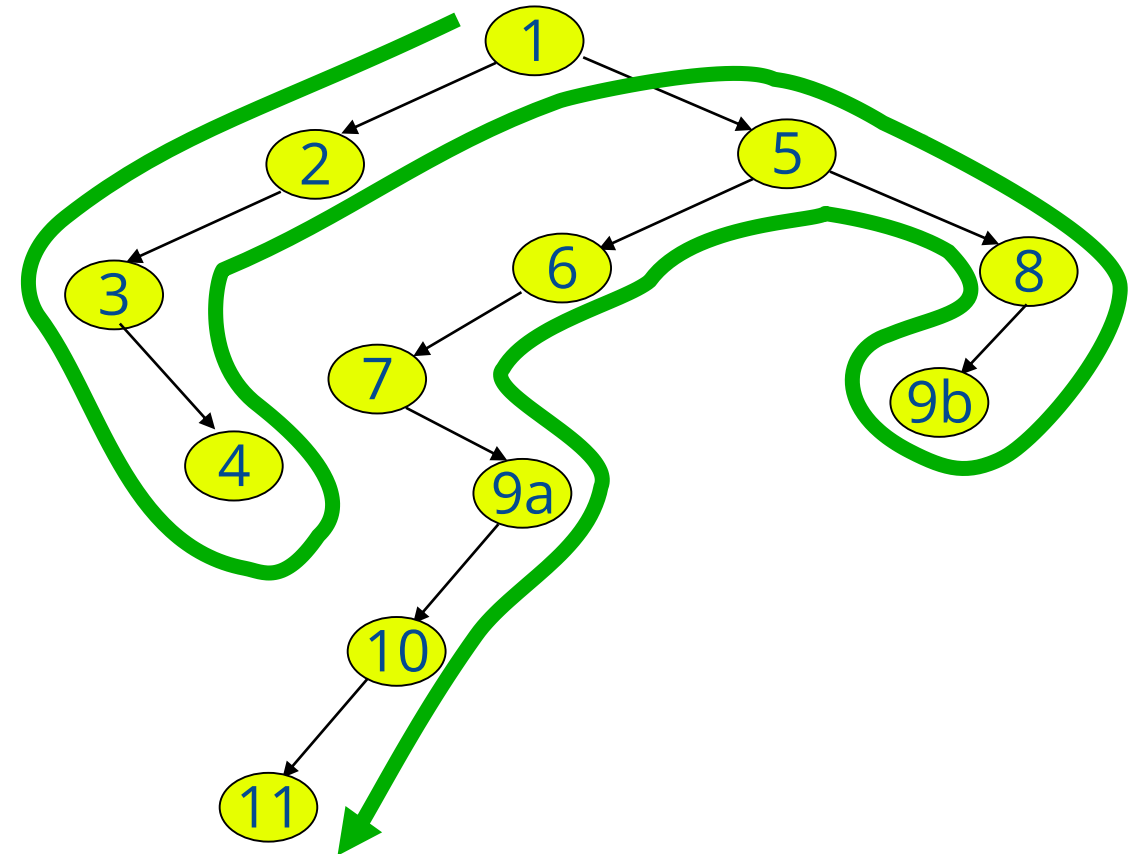
BSP Trees

Tree traversal

- If e is outside (or in front of) a face i
 - Draw everything behind i , Draw i , Draw everything in front of i
- If e is inside a (or behind) face i
 - Draw everything in front of i , Draw i , Draw everything behind i



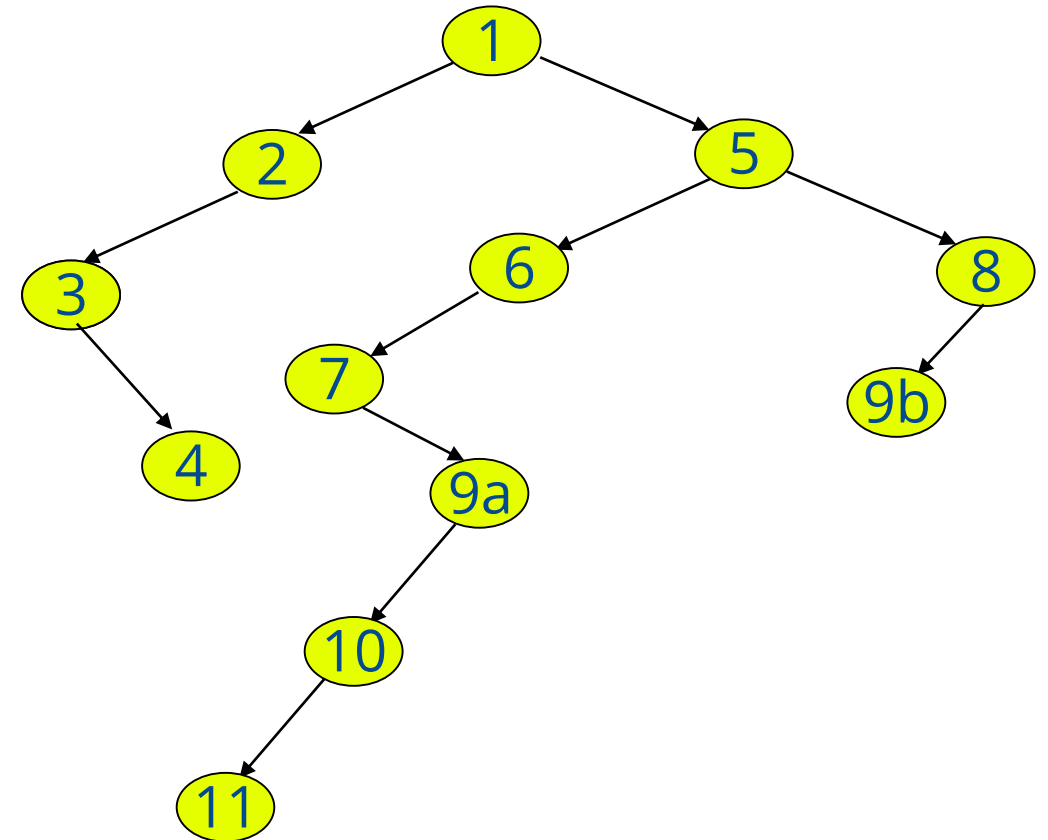
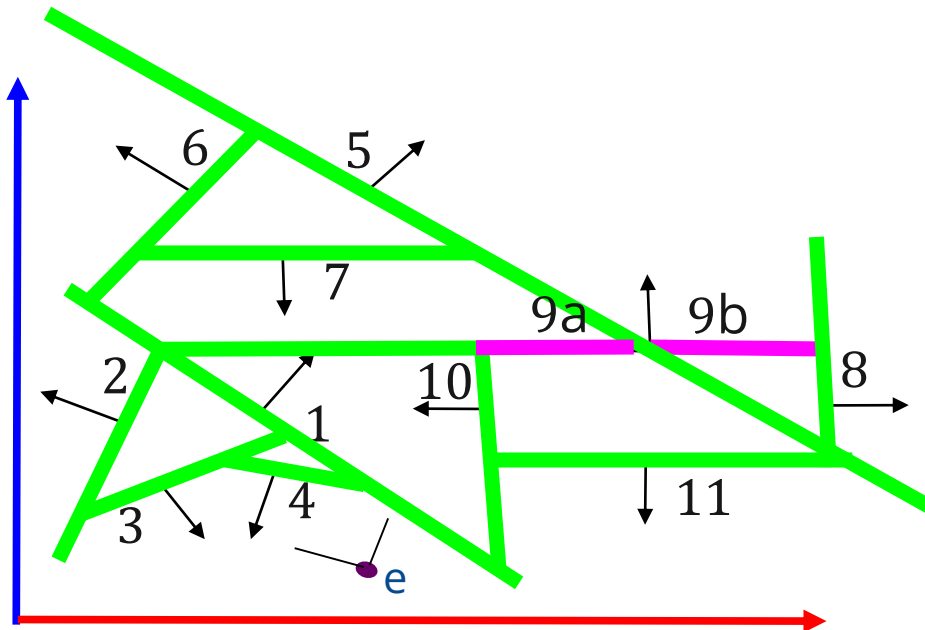
2, 3, 4, 1, 8, 9b, 5, 6, 7, 9a, 10, 11



BSP Trees

Tree traversal

- If e is outside (or in front of) a face i
 - Draw everything behind i , Draw i , Draw everything in front of i
- If e is inside a (or behind) face i
 - Draw everything in front of i , Draw i , Draw everything behind i

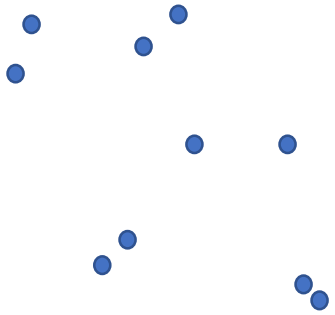


What is the traversal order now?

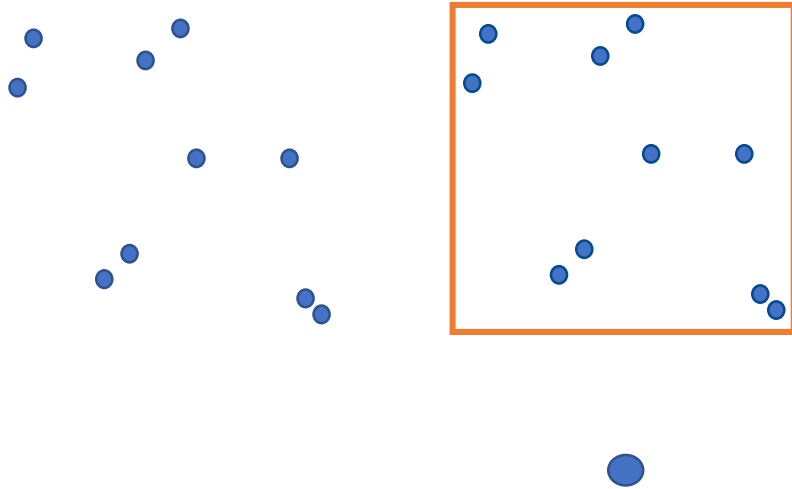
Other space partitioning schemes

- Split into equal parts along the axis
 - Quadtrees
 - 2D, always divide a bounding box into 4 parts
 - Storing sparse data, image compression
 - Octrees
 - 3D, always divide a bounding box into 8 parts
 - Used for view-frustum culling, storing levels of detail (LOD)
- Split along the axis through the median
 - K-d tree
 - Generalizes to any dimension
 - Performance degrades with higher dimensions
 - Used for approximate nearest neighbour searches

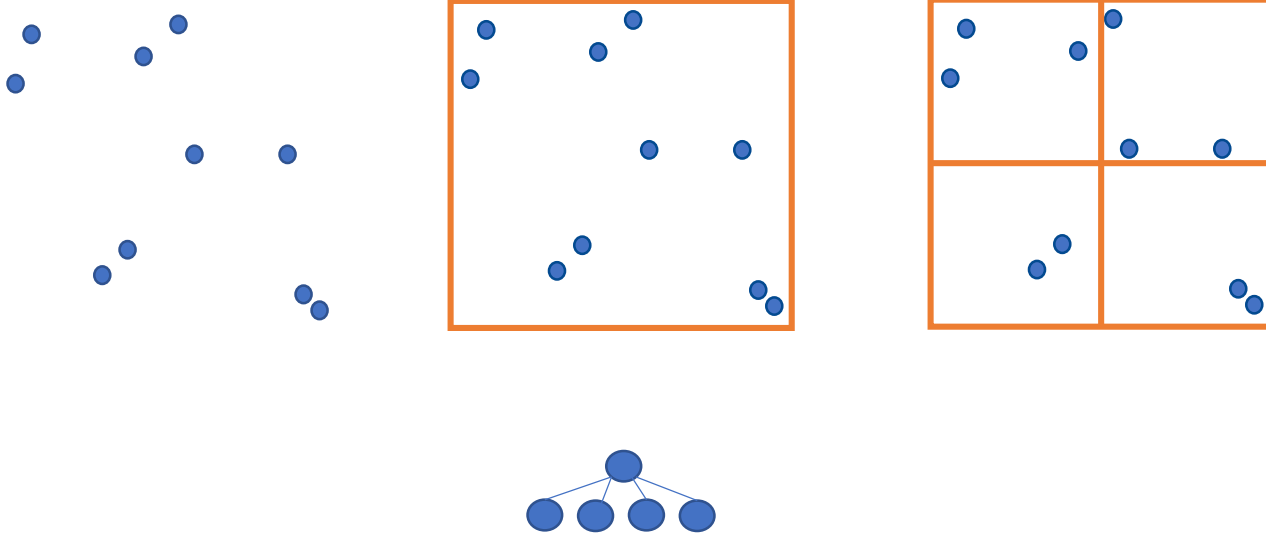
Quadrees and Octrees



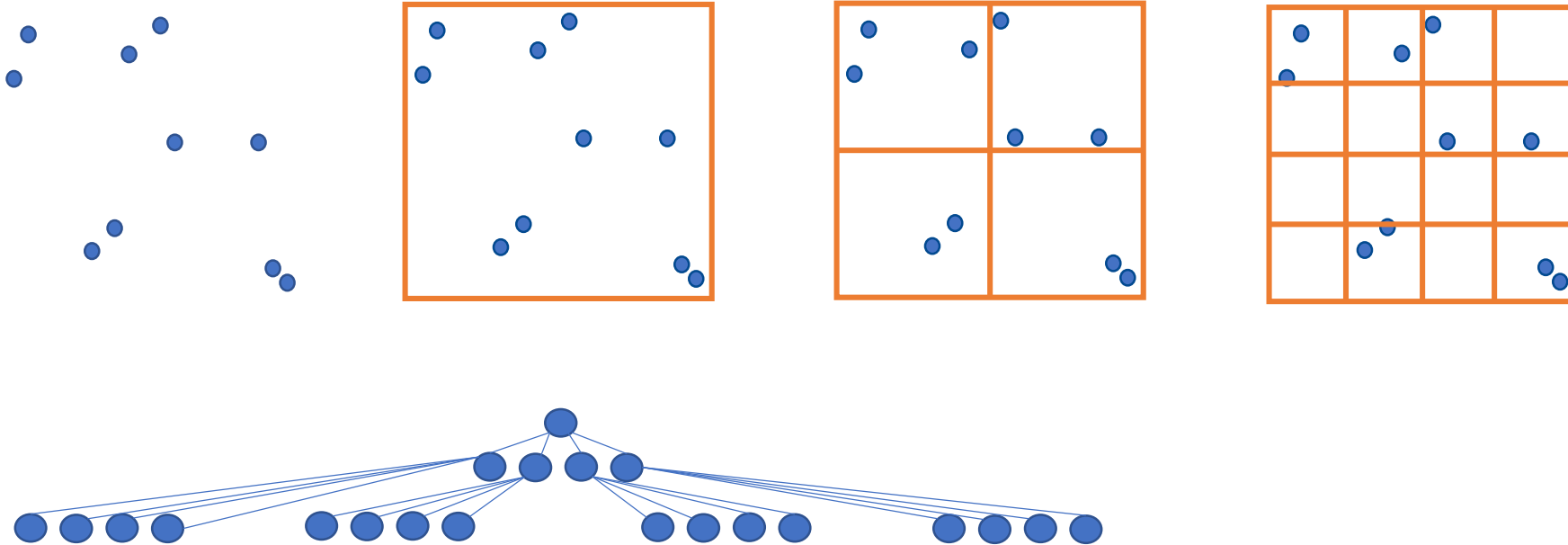
Quadrees and Octrees



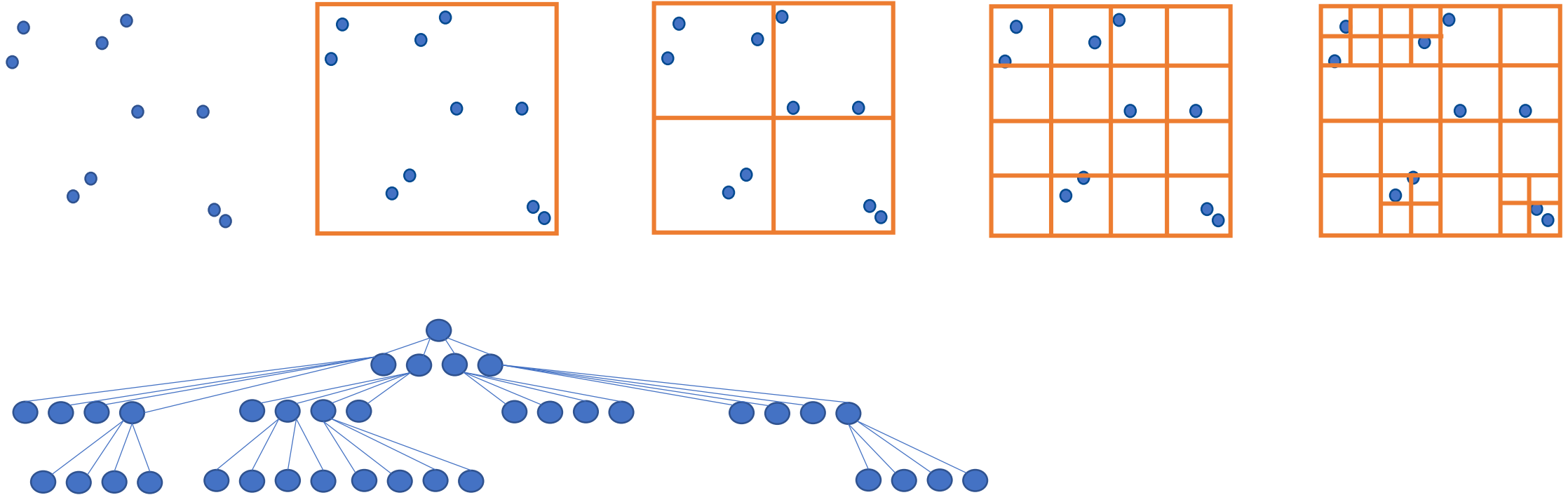
Quadrees and Octrees



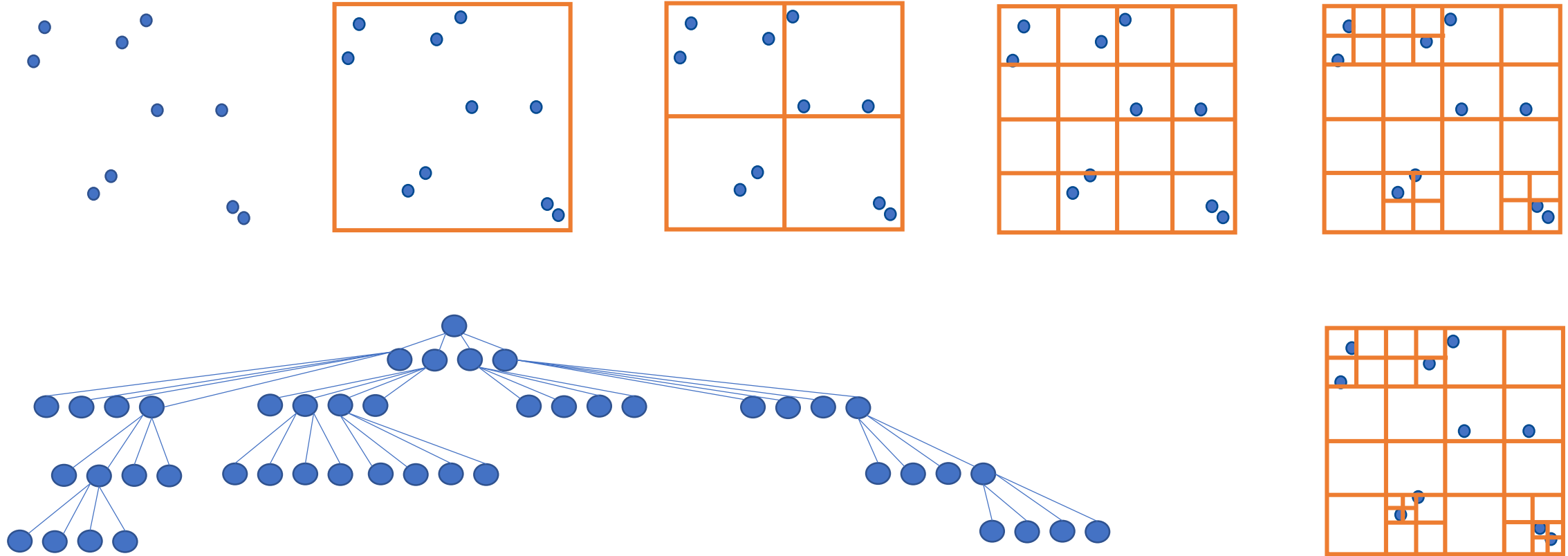
Quadtrees and Octrees



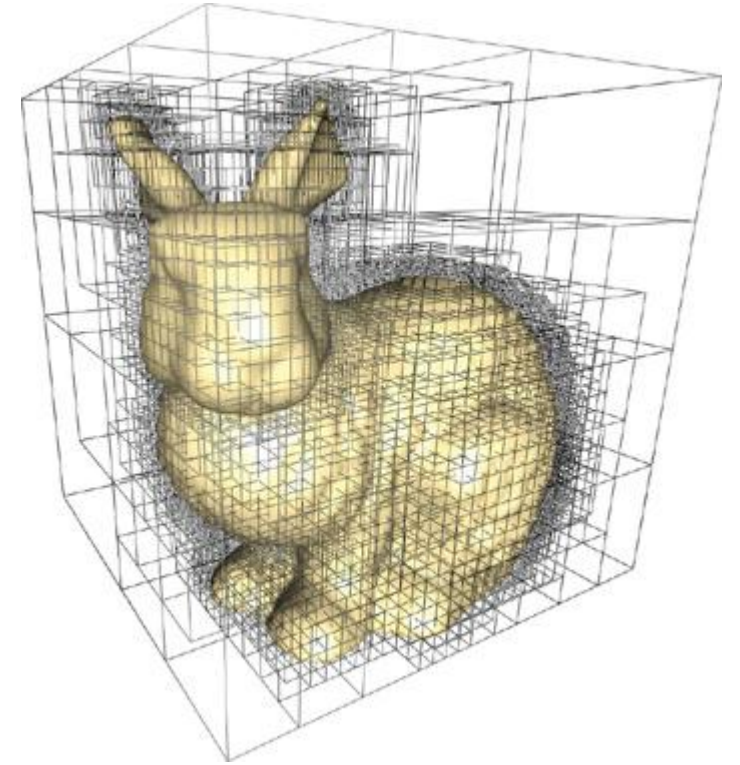
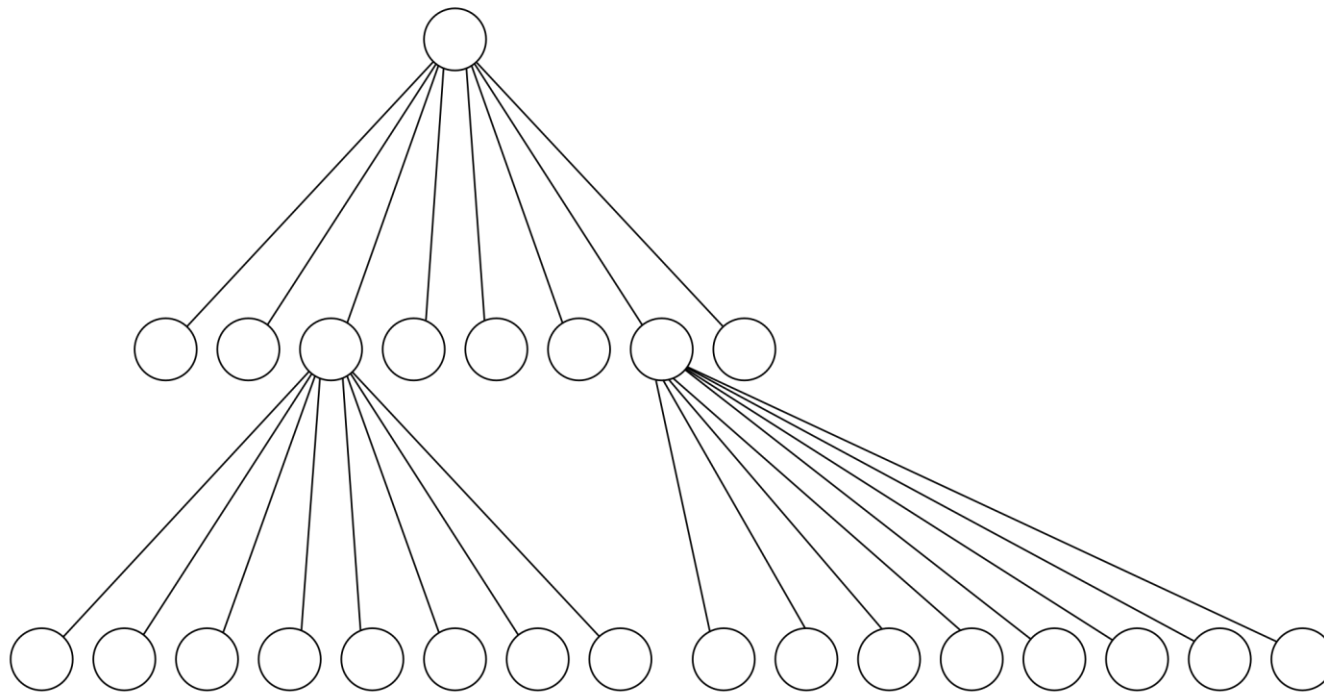
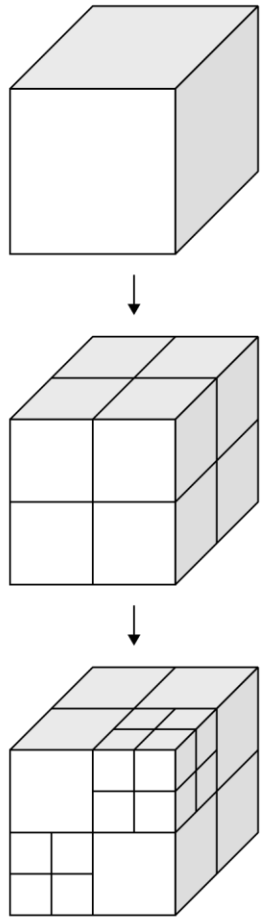
Quadrees and Octrees



Quadtrees and Octrees

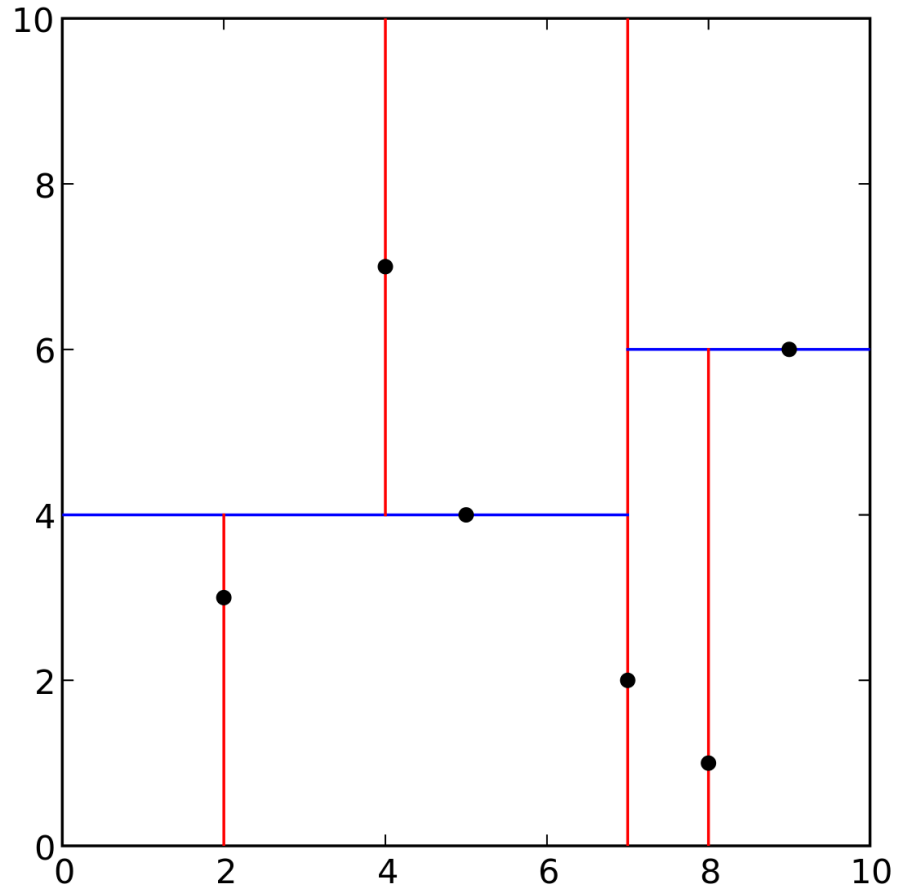


Quadrees and Octrees



<https://developer.nvidia.com/gpugems/gpugems2/part-v-image-oriented-computing/chapter-37-octree-textures-gpu>

k-d Tree



https://en.wikipedia.org/wiki/K-d_tree

k-d tree decomposition for the point set (2,3), (5,4), (9,6), (4,7), (8,1), (7,2).

